

VOLUME 1 — FUNDAMENTOS DA IA GENERATIVA

CAPÍTULO 3

Arquitetura Transformer

O mecanismo de atenção que tornou os LLMs modernos possíveis

🎯 Objetivos de Aprendizagem

- Explicar por que redes neurais recorrentes (RNNs) eram limitadas para processar texto longo.
- Descrever o que é tokenização e por que modelos de linguagem não leem 'palavras' diretamente.
- Entender o mecanismo de self-attention e o papel dos vetores Query, Key e Value.
- Explicar por que a atenção multi-cabeça e a codificação posicional são necessárias.
- Reconhecer a arquitetura completa do Transformer (encoder, decoder, blocos residuais).
- Implementar, em Python puro, uma versão simplificada de self-attention e aplicá-la ao Assistente Aurora.

1. Introdução

No Capítulo 2, vimos como redes neurais aprendem ajustando pesos via gradiente descendente, e como empilhar camadas cria modelos capazes de aprender representações cada vez mais abstratas. Mas ficou uma pergunta em aberto: como uma rede neural processa texto, que é uma sequência de tamanho variável, em que o significado de uma palavra depende de palavras distantes na frase? Este capítulo responde a essa pergunta apresentando a arquitetura Transformer, introduzida em 2017 no artigo “Attention Is All You Need”, da equipe do Google — a peça de engenharia que sustenta praticamente todo LLM relevante hoje, do GPT ao modelo que você usará no Volume 4 para construir seu sistema RAG.

Vamos construir o entendimento em camadas: primeiro o problema que motivou o Transformer, depois seus blocos constituintes (tokenização, embeddings, atenção, codificação posicional), e por fim como esses blocos se encaixam na arquitetura completa. Este capítulo é deliberadamente conceitual e visual — o Volume 2 vai aprofundar embeddings, e o Volume 5 vai mostrar como usar Transformers prontos via bibliotecas como Hugging Face e LangChain, sem que você precise implementá-los do zero em produção. Mas entender o mecanismo interno é o que separa quem apenas usa uma API de quem sabe por que ela se comporta de determinada forma.

2. O problema que o Transformer resolveu

2.1 Redes Neurais Recorrentes (RNNs): processando texto palavra por palavra

Antes de 2017, o padrão para processar sequências (como texto ou áudio) eram as Redes Neurais Recorrentes (RNNs) e suas variantes mais sofisticadas, como LSTM e GRU. A ideia central: processar a sequência um elemento de cada vez, mantendo um “estado oculto” que resume tudo o que foi visto até aquele ponto, e atualizando esse estado a cada nova palavra.

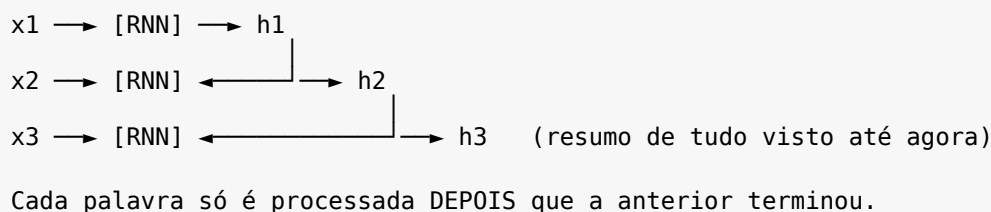


Figura 3.1 — Processamento sequencial em uma RNN: uma palavra de cada vez.

2.2 Duas limitações fundamentais

Essa abordagem sequencial trazia dois problemas sérios. O primeiro é de desempenho: como cada passo depende do resultado do passo anterior, é impossível paralelizar o processamento de uma sequência inteira — mesmo com uma GPU poderosa, capaz de fazer milhares de cálculos simultâneos, uma RNN é forçada a processar palavra após palavra, sequencialmente. Em textos longos, isso torna o treinamento extremamente lento.

O segundo problema é de memória efetiva: o estado oculto é um vetor de tamanho fixo que precisa resumir tudo o que veio antes. Quanto mais distante uma informação relevante está no texto, mais ela se dilui nesse resumo — um fenômeno relacionado ao “gradiente desaparecendo” mencionado no Capítulo 2. Na prática, RNNs tinham dificuldade genuína em conectar uma palavra no início de um parágrafo longo com uma palavra relacionada no final.

Analogia

Imagine ler um romance de 400 páginas anotando, a cada página, um resumo de uma frase de tudo o que leu até ali, sem poder voltar as páginas depois. Ao chegar no capítulo final, seu “resumo acumulado” provavelmente já perdeu detalhes específicos do capítulo 2 — mesmo que esses detalhes sejam cruciais para entender o desfecho. É exatamente essa perda progressiva de detalhes distantes que a arquitetura Transformer resolve, permitindo que o modelo “olhe para trás”, em qualquer ponto do texto, diretamente — sem depender de um resumo comprimido.

3. Preparando o texto: tokenização e embeddings de entrada

3.1 Tokenização: do texto a números

Redes neurais só operam sobre números, nunca sobre texto bruto. O primeiro passo de qualquer Transformer é a tokenização: quebrar o texto em unidades menores chamadas tokens, e mapear cada token para um número inteiro (seu identificador em um vocabulário fixo). É um erro comum assumir que um token é sempre uma palavra inteira — na prática, a maioria dos LLMs modernos usa tokenização por subpalavras (como o algoritmo Byte-Pair Encoding, BPE), em que palavras raras ou compostas são quebradas em pedaços menores e mais frequentes.

```
>>> # Exemplo ilustrativo de tokenização por subpalavras
>>> texto = "transformação"
>>> tokens = ["transform", "ação"]      # duas subpalavras, não uma
>>> ids = [4821, 902]                  # cada subpalavra vira um número

>>> texto2 = "gato"
>>> tokens2 = ["gato"]                 # palavra comum: um único token
>>> ids2 = [318]
```

Código 3.1 — Ilustração conceitual de tokenização por subpalavras (BPE).

Essa escolha de projeto explica um comportamento conhecido dos LLMs: eles às vezes erram em tarefas simples como contar letras de uma palavra, porque não “veem” letras individuais — veem tokens, que frequentemente agrupam várias letras. O Volume 5 vai mostrar como inspecionar a tokenização real de modelos como GPT usando bibliotecas específicas.

3.2 Embeddings de entrada: de número a vetor de significado

Cada identificador de token é convertido em um vetor de números reais — uma lista de, tipicamente, algumas centenas ou milhares de números — chamado embedding. Esse vetor é aprendido durante o treinamento e captura relações de significado: tokens usados em contextos parecidos acabam com vetores numericamente próximos. Este é apenas um vislumbre do assunto: o Volume 2 inteiro é dedicado a explicar como esses vetores são treinados, como medir “proximidade” entre eles, e como usá-los para busca semântica — a base de todo sistema RAG.

4. O coração do Transformer: self-attention

4.1 A ideia central

Self-attention (atenção sobre si mesma) permite que, ao processar uma palavra, o modelo “olhe” diretamente para todas as outras palavras da sequência — inclusive as muito distantes — e decida, de forma aprendida, quais delas são relevantes para interpretar a palavra atual. Diferente da RNN, esse processo acontece em paralelo para todas as posições da sequência ao mesmo tempo, o que resolve o problema de desempenho, e sem perda de informação distante, o que resolve o problema de memória.

Analogia

Pense em um tradutor humano traduzindo a palavra “ela” em uma frase longa. Para saber a quem “ela” se refere, o tradutor não confia em um resumo vago da frase inteira — ele olha diretamente para trás, encontra o substantivo correspondente (“a engenheira”, por exemplo), e usa essa informação específica. A self-attention automatiza exatamente esse tipo de busca direcionada, para cada palavra da sequência, simultaneamente.

4.2 Query, Key e Value: a analogia do sistema de busca

Tecnicamente, a self-attention transforma cada vetor de entrada em três vetores diferentes, através de três matrizes de pesos aprendidas durante o treino: Query (Q, “consulta” — o que esta posição está procurando), Key (K, “chave” — o que cada posição tem a oferecer, usado para comparação) e Value (V, “valor” — a informação real que será agregada, caso a posição seja considerada relevante).

Analogia

É útil pensar nisso como uma busca em uma biblioteca digital. Sua pesquisa é o Query (“preciso de artigos sobre gradiente descendente”). Cada livro da biblioteca tem uma etiqueta de metadados — o Key (“este livro é sobre otimização matemática”). O sistema compara sua consulta com as etiquetas de todos os livros para calcular uma pontuação de relevância, e então devolve o conteúdo — o Value — dos livros mais relevantes, ponderado pela pontuação de cada um. Essa mesma lógica de “Query compara com Key para decidir quanto usar de cada Value” é exatamente o que faremos, de forma muito mais explícita e literal, com embeddings e busca vetorial no Volume 2 e no Volume 3.

4.3 O cálculo, passo a passo

Em termos simplificados, a self-attention para uma posição calcula: (1) o produto escalar entre o Query dessa posição e o Key de cada outra posição da sequência, produzindo uma pontuação de similaridade para cada par; (2) normaliza essas pontuações com uma função softmax, transformando-as em pesos que somam 1 (uma distribuição de “quanto prestar atenção” em cada posição); (3) usa esses pesos para fazer uma média ponderada dos vetores Value de todas as posições. O resultado é um novo vetor para aquela posição, que já incorpora informação relevante do resto da sequência.

Frase: 'O gato dormiu porque ele estava cansado'

Ao processar 'ele', a atenção calcula pesos para cada palavra:

0	gato	dormiu	porque	ele	estava	cansado
0.02	0.71	0.05	0.03	-	0.04	0.15

▲
maior peso: o modelo aprendeu a associar
'ele' fortemente a 'gato' nesta frase.

Figura 3.2 — Pesos de atenção ilustrativos ao processar a palavra 'ele'.

5. Atenção multi-cabeça e codificação posicional

5.1 Por que várias 'cabeças' de atenção?

Uma única operação de self-attention aprende um único tipo de relação entre palavras. Mas a linguagem tem múltiplas dimensões de relação simultâneas: relação gramatical (sujeito-verbo), relação semântica (sinônimos, antônimos), relação referencial (pronomes e seus referentes), entre outras. A atenção multi-cabeça (multi-head attention) resolve isso executando várias operações de self-attention em paralelo — cada “cabeça” com suas próprias matrizes Q, K e V aprendidas — e depois combinando os resultados. Na prática, diferentes cabeças frequentemente se especializam em diferentes tipos de padrão linguístico, embora ninguém programe essa especialização explicitamente; ela emerge do treino.

5.2 Codificação posicional: devolvendo a noção de ordem

Há um detalhe importante que passamos por cima: a operação de self-attention, por si só, é invariante à ordem das palavras — ela calcula relações entre pares de posições, mas não “sabe” nativamente que “gato” apareceu antes de “dormiu”. Isso é um problema óbvio para linguagem, onde a ordem das palavras importa (“o cão mordeu o homem” é bem diferente de “o homem mordeu o cão”).

A solução do Transformer é a codificação posicional (positional encoding): antes de entrar nas camadas de atenção, um vetor que representa a posição de cada token na sequência (usando funções seno e cosseno de frequências diferentes, no artigo original) é somado ao embedding daquele token. Assim, o próprio vetor de entrada já carrega informação de “onde” aquele token está na sequência, sem que o mecanismo de atenção precise processar a sequência em ordem.

6. A arquitetura completa: encoder, decoder e blocos residuais

O artigo original de 2017 descreve uma arquitetura encoder-decoder, pensada originalmente para tradução automática: o encoder lê a frase de entrada inteira e produz uma representação rica dela; o decoder gera a frase

de saída, token a token, prestando atenção tanto ao que já gerou quanto à representação produzida pelo encoder. Modelos como o BERT usam apenas o encoder (bom para entender texto), enquanto modelos como o GPT usam apenas o decoder (bom para gerar texto) — essa distinção será relevante quando escolhermos modelos para diferentes tarefas no Volume 5.

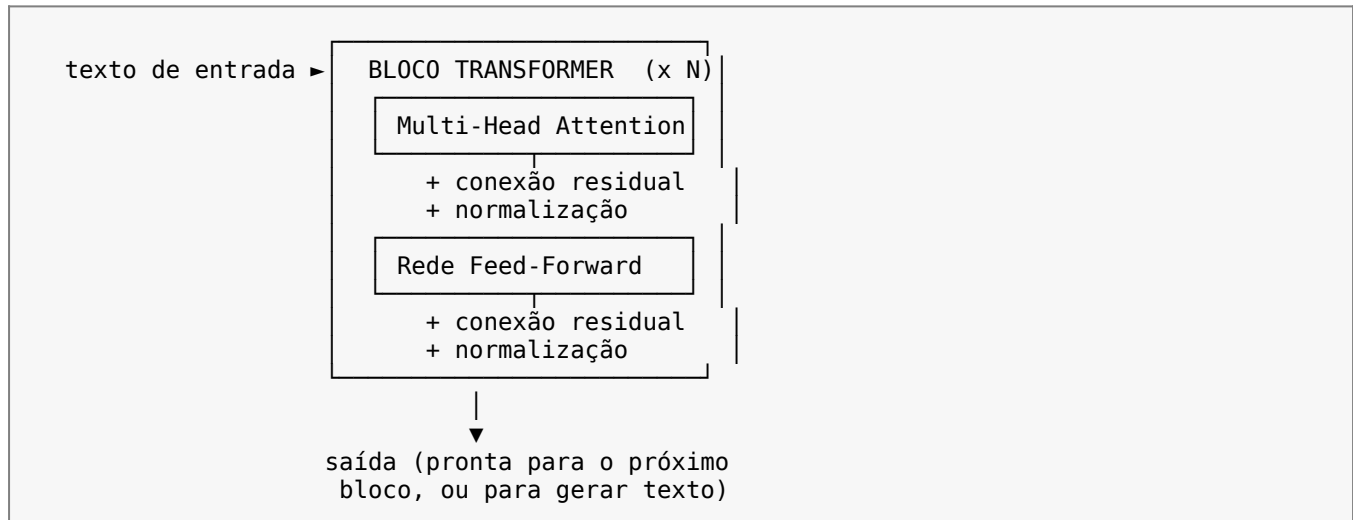


Figura 3.3 — Um bloco Transformer: atenção multi-cabeça + rede feed-forward, com conexões residuais.

Dois detalhes de engenharia merecem destaque. As conexões residuais (residual connections) somam a entrada de cada sub-bloco diretamente à sua saída, o que ajuda o gradiente a fluir por redes muito profundas sem se dissipar — um problema que mencionamos no Capítulo 2 ao falar de funções de ativação. A normalização de camada (layer normalization) reescala os valores internos para manter o treinamento numericamente estável. Um LLM real empilha dezenas desses blocos — o GPT-3, por exemplo, usa 96 blocos — cada um refinando progressivamente a representação do texto.

7. Implementando self-attention simplificada em Python

Vamos implementar uma versão simplificada e didática de self-attention usando apenas NumPy, sem nenhuma biblioteca de Deep Learning — o objetivo é tornar tangível a matemática descrita na Seção 4.3, não construir um Transformer pronto para produção (isso virá pronto de bibliotecas no Volume 5).

```

# self_attention_simplificada.py
# Implementação didática de self-attention com NumPy puro.
import numpy as np

def softmax(x):
    """Transforma um vetor de pontuações em uma distribuição que soma 1."""
    exp_x = np.exp(x - np.max(x)) # subtrai o máximo por estabilidade numérica
    return exp_x / exp_x.sum(axis=-1, keepdims=True)

def self_attention(embeddings, W_q, W_k, W_v):
    """
    embeddings: matriz (num_tokens, dim_embedding) — um vetor por token.
    W_q, W_k, W_v: matrizes de pesos aprendidas (dim_embedding, dim_embedding).
    """
    # 1. Projeta os embeddings em Query, Key e Value.
    Q = embeddings @ W_q
    K = embeddings @ W_k
    V = embeddings @ W_v
  
```

```

# 2. Calcula a similaridade entre cada Query e cada Key (produto escalar).
dim = embeddings.shape[1]
pontuacoes = (Q @ K.T) / np.sqrt(dim) # divisão estabiliza os valores

# 3. Normaliza as pontuações em pesos de atenção (somam 1 por linha).
pesos_atencao = softmax(pontuacoes)

# 4. Combina os Values ponderados pelos pesos de atenção.
saida = pesos_atencao @ V
return saida, pesos_atencao

# --- Exemplo com 4 tokens e embeddings de dimensão 3 (valores fictícios) ---
np.random.seed(42)
embeddings = np.random.rand(4, 3) # 4 tokens: 'o', 'gato', 'dormiu', 'cansado'
W_q = np.random.rand(3, 3)
W_k = np.random.rand(3, 3)
W_v = np.random.rand(3, 3)

saida, pesos = self_attention(embeddings, W_q, W_k, W_v)
print("Pesos de atenção (cada linha soma 1):")
print(np.round(pesos, 2))

```

Código 3.2 — Self-attention implementada do zero com NumPy.

Rodando esse código, cada linha da matriz `pesos_atencao` mostra, para um token, o quanto ele “presta atenção” em cada um dos outros tokens (incluindo ele mesmo) — exatamente a ideia ilustrada na Figura 3.2, só que com números reais calculados, em vez de valores fictícios para ilustração. Em um Transformer de verdade, `W_q`, `W_k` e `W_v` não são aleatórias: são aprendidas via gradiente descendente durante o treinamento, exatamente como os pesos do MLP do Capítulo 2.

8. Projeto Prático Incremental: Assistente Aurora

Nos capítulos anteriores, Aurora ganhou um esqueleto (Capítulo 1) e um classificador de intenções baseado em rede neural (Capítulo 2). Neste capítulo, ainda não vamos plugar um Transformer completo em Aurora — isso é prematuro sem embeddings de verdade, que chegam no Volume 2. Em vez disso, vamos adicionar um módulo pedagógico de inspeção de atenção, que a Aurora poderá usar para 'mostrar seu raciocínio' futuramente, e um tokenizador simples que prepara terreno para o Volume 2.



Projeto: Assistente Aurora — Parte 3 — Tokenizador simples e inspeção de atenção

Nesta parte você vai:

- Criar um tokenizador simples baseado em espaços e pontuação (uma aproximação didática).
- Adicionar à Aurora uma função que calcula e expõe pesos de atenção fictícios para fins de depuração.
- Preparar a estrutura que será substituída por tokenização e embeddings reais no Volume 2.

```

# aurora/tokenizador.py
# Tokenizador simplificado — versão didática (Volume 2 traz um tokenizador real).
import re

class TokenizadorSimples:
    """Quebra texto em tokens de palavras e pontuação, em minúsculas."""

```

```
def tokenizar(self, texto: str) -> list[str]:
    texto = texto.lower()
    # Separa palavras de pontuação usando uma expressão regular simples.
    tokens = re.findall(r"\w+|[.,!?:;]", texto)
    return tokens

if __name__ == "__main__":
    tok = TokenizadorSimples()
    print(tok.tokenizar("O gato dormiu, porque ele estava cansado!"))
    # ['o', 'gato', 'dormiu', ',', 'porque', 'ele', 'estava', 'cansado', '!']
```

Código 3.3 — *aurora/tokenizador.py*, uma primeira aproximação de tokenização.

```
# aurora/inspecao_atencao.py
# Módulo de depuração: mostra 'para onde a Aurora estaria olhando' em uma frase.
# Usa a self_attention do Código 3.2 com embeddings ALEATÓRIOS apenas para fins
# didáticos – no Volume 2, substituiremos por embeddings de significado reais.
import numpy as np
from .tokenizador import TokenizadorSimples

def inspecionar_atencao(frase: str, dim_embedding: int = 8):
    tokenizador = TokenizadorSimples()
    tokens = tokenizador.tokenizar(frase)

    np.random.seed(0)
    embeddings = np.random.rand(len(tokens), dim_embedding)
    W_q = np.random.rand(dim_embedding, dim_embedding)
    W_k = np.random.rand(dim_embedding, dim_embedding)
    W_v = np.random.rand(dim_embedding, dim_embedding)

    Q, K = embeddings @ W_q, embeddings @ W_k
    pontuacoes = (Q @ K.T) / np.sqrt(dim_embedding)
    exp = np.exp(pontuacoes - pontuacoes.max(axis=-1, keepdims=True))
    pesos = exp / exp.sum(axis=-1, keepdims=True)

    for i, token in enumerate(tokens):
        maior_indice = int(np.argmax(pesos[i]))
        print(f"'{token}' presta mais atenção em: '{tokens[maior_indice]}'")

    return tokens, pesos

if __name__ == "__main__":
    inspecionar_atencao("O gato dormiu porque ele estava cansado")
```

Código 3.4 — *aurora/inspecao_atencao.py*, ferramenta de depuração pedagógica.

É importante deixar claro: como os embeddings aqui são aleatórios (ainda não aprendidos), os pesos de atenção não terão significado linguístico real — o valor deste módulo é puramente didático, para você visualizar o mecanismo. No Volume 2, ao introduzir embeddings de significado real (pré-treinados), esse mesmo código de atenção passará a produzir padrões que fazem sentido linguístico, como na Figura 3.2.

9. Exercícios Resolvidos

Exercício Resolvido 1

Modifique a função `self_attention` do Código 3.2 para retornar também a pontuação bruta (antes do softmax), de forma que seja possível inspecionar os valores antes da normalização.

Solução:

```
def self_attention_com_pontuacoes(embeddings, W_q, W_k, W_v):
    Q = embeddings @ W_q
    K = embeddings @ W_k
    V = embeddings @ W_v

    dim = embeddings.shape[1]
    pontuacoes_brutas = (Q @ K.T) / np.sqrt(dim)

    exp = np.exp(pontuacoes_brutas - pontuacoes_brutas.max(axis=-1, keepdims=True))
    pesos_atencao = exp / exp.sum(axis=-1, keepdims=True)

    saida = pesos_atencao @ V
    return saida, pesos_atencao, pontuacoes_brutas # NOVO: retorno extra
```

Exercício Resolvido 2

Explique, usando o TokenizadorSimples do Código 3.3, por que a frase 'não-sei' geraria um único token com a expressão regular `\w+`, e proponha um ajuste para separar o hífen.

Solução:

```
# Com \w+, o hífen NÃO é capturado por \w (que só pega letras, números e underscore),
# então 'não-sei' seria dividido em dois tokens: 'não' e 'sei', e o hífen é descartado
# (não aparece como token próprio, diferente da vírgula ou ponto de exclamação).

# Para tratar o hífen como token próprio (por exemplo, para preservar pontuação
# composta), ajustamos a expressão regular:
import re
tokens = re.findall(r"\w+|[\.,!?:;-]", "não-sei".lower())
print(tokens) # ['não', '-', 'sei']
```

10. Exercícios Propostos

1. Explique, com suas próprias palavras, por que a self-attention é mais fácil de paralelizar em GPU do que o processamento de uma RNN.
2. No Código 3.2, aumente o número de tokens para 6 e observe se a matriz de pesos de atenção continua tendo cada linha somando 1 (dica: use `pesos.sum(axis=1)`).
3. Pesquise a diferença entre um modelo 'somente encoder' (como o BERT) e um modelo 'somente decoder' (como o GPT), e explique em que tipo de tarefa cada um costuma ser mais indicado.
4. Implemente uma versão da função `inspecionar_atencao` do Código 3.4 que simule 2 cabeças de atenção (2 conjuntos diferentes de `W_q`, `W_k`, `W_v`) e compare os resultados.
5. Explique por que a codificação posicional é somada ao embedding, e não concatenada — pesquise as implicações de cada abordagem, se preferir se aprofundar.

11. Quiz do Capítulo

Escolha a alternativa correta para cada pergunta. O gabarito está ao final da seção.

1. Qual é a principal limitação de desempenho das RNNs ao processar sequências longas?

- A) Elas não conseguem rodar em CPU

- B) O processamento é sequencial e não pode ser paralelizado
- C) Elas não usam funções de ativação
- D) Elas exigem mais memória RAM que Transformers

2. O que é um token, no contexto de LLMs?

- A) Sempre uma palavra inteira
- B) Uma unidade de texto (palavra, subpalavra ou símbolo) mapeada para um número
- C) Um parâmetro do modelo
- D) Uma camada da rede neural

3. No mecanismo de self-attention, para que serve o vetor Query?

- A) Representa o que uma posição está 'procurando' na sequência
- B) Armazena o resultado final da atenção
- C) Define a taxa de aprendizado
- D) Substitui a função de ativação

4. Por que a codificação posicional é necessária no Transformer?

- A) Porque o texto sempre tem tamanho fixo
- B) Porque a self-attention, por si só, não tem noção de ordem das posições
- C) Porque acelera o treinamento em GPU
- D) Porque substitui a tokenização

5. Qual a vantagem da atenção multi-cabeça sobre uma única cabeça de atenção?

- A) Reduz o número de parâmetros do modelo
- B) Permite capturar diferentes tipos de relação linguística em paralelo
- C) Elimina a necessidade de embeddings
- D) Torna o modelo determinístico

6. O que a função softmax faz sobre as pontuações de atenção?

- A) Multiplica todas por zero
- B) Transforma as pontuações em pesos que somam 1
- C) Remove os tokens irrelevantes do vocabulário
- D) Converte texto em tokens

7. Para que servem as conexões residuais em um bloco Transformer?

- A) Aumentar a velocidade de tokenização
- B) Ajudar o gradiente a fluir por redes profundas sem se dissipar
- C) Substituir a atenção multi-cabeça
- D) Reduzir o tamanho do vocabulário

8. Modelos como o BERT usam qual parte da arquitetura original encoder-decoder?

- A) Apenas o decoder
- B) Apenas o encoder
- C) Encoder e decoder completos
- D) Nenhuma das duas

9. No Código 3.2, por que a pontuação de atenção é dividida por np.sqrt(dim) ?

- A) Para acelerar o cálculo em GPU
- B) Para estabilizar numericamente os valores antes do softmax
- C) Para gerar números aleatórios
- D) Para remover tokens duplicados

10. Por que os embeddings usados no projeto Aurora (Código 3.4) ainda não produzem atenção linguisticamente significativa?

- A) Porque o tokenizador está com erro
- B) Porque os embeddings são gerados aleatoriamente, não aprendidos
- C) Porque faltam conexões residuais
- D) Porque o softmax não foi aplicado

Gabarito

1B · 2B · 3A · 4B · 5B · 6B · 7B · 8B · 9B · 10B

12. Resumo do Capítulo

- RNNs processam texto sequencialmente, o que limita paralelização e dificulta capturar dependências distantes.
- Tokenização quebra o texto em unidades (tokens), frequentemente subpalavras, mapeadas para números.
- Embeddings transformam tokens em vetores que capturam significado, aprendidos durante o treinamento.
- Self-attention usa vetores Query, Key e Value para permitir que cada token 'olhe' diretamente para qualquer outro token da sequência.
- Atenção multi-cabeça executa várias operações de atenção em paralelo, capturando diferentes tipos de relação linguística.
- Codificação posicional devolve a noção de ordem que a self-attention, por si só, não possui.
- Um bloco Transformer combina atenção multi-cabeça e uma rede feed-forward, com conexões residuais e normalização; modelos reais empilham dezenas desses blocos.
- Aurora ganhou um tokenizador simples e um módulo de inspeção de atenção, preparando o terreno para embeddings reais no Volume 2.

13. Glossário

Atenção multi-cabeça (multi-head attention): Execução paralela de várias operações de self-attention, cada uma com pesos próprios, para capturar diferentes tipos de relação.

Codificação posicional (positional encoding): Vetor somado ao embedding de cada token para indicar sua posição na sequência, já que a atenção não possui noção nativa de ordem.

Conexão residual: Técnica que soma a entrada de um sub-bloco à sua saída, ajudando o gradiente a fluir em redes profundas.

Decoder: Parte da arquitetura Transformer responsável por gerar a sequência de saída token a token; base de modelos como o GPT.

Encoder: Parte da arquitetura Transformer responsável por processar e representar a sequência de entrada; base de modelos como o BERT.

Query, Key, Value (Q, K, V): Três vetores derivados de cada token na self-attention: Query representa o que se busca, Key o que se oferece para comparação, Value a informação a ser agregada.

Self-attention: Mecanismo que permite a cada posição de uma sequência calcular sua relação direta com todas as outras posições, em paralelo.

Softmax: Função matemática que converte um conjunto de pontuações em uma distribuição de probabilidade que soma 1.

Token: Unidade mínima de texto (palavra, subpalavra ou símbolo) usada como entrada por um modelo de linguagem.

Tokenização: Processo de dividir um texto em tokens e mapeá-los para identificadores numéricos de um vocabulário.